

Chapitre I

PL/SQL

Partie 1

Introduction

F. BARIKA KTATA

- SQL est un langage complet pour travailler sur une base de données.
 - SQL ne comporte pas d'instruction procédurale.
- ➔ Ce qui a donnée naissance à un langage plus complet qui est le **PL/SQL** (**P**rocedura**L** / **S**tructred **Q**uery **L**angage)

Introduction

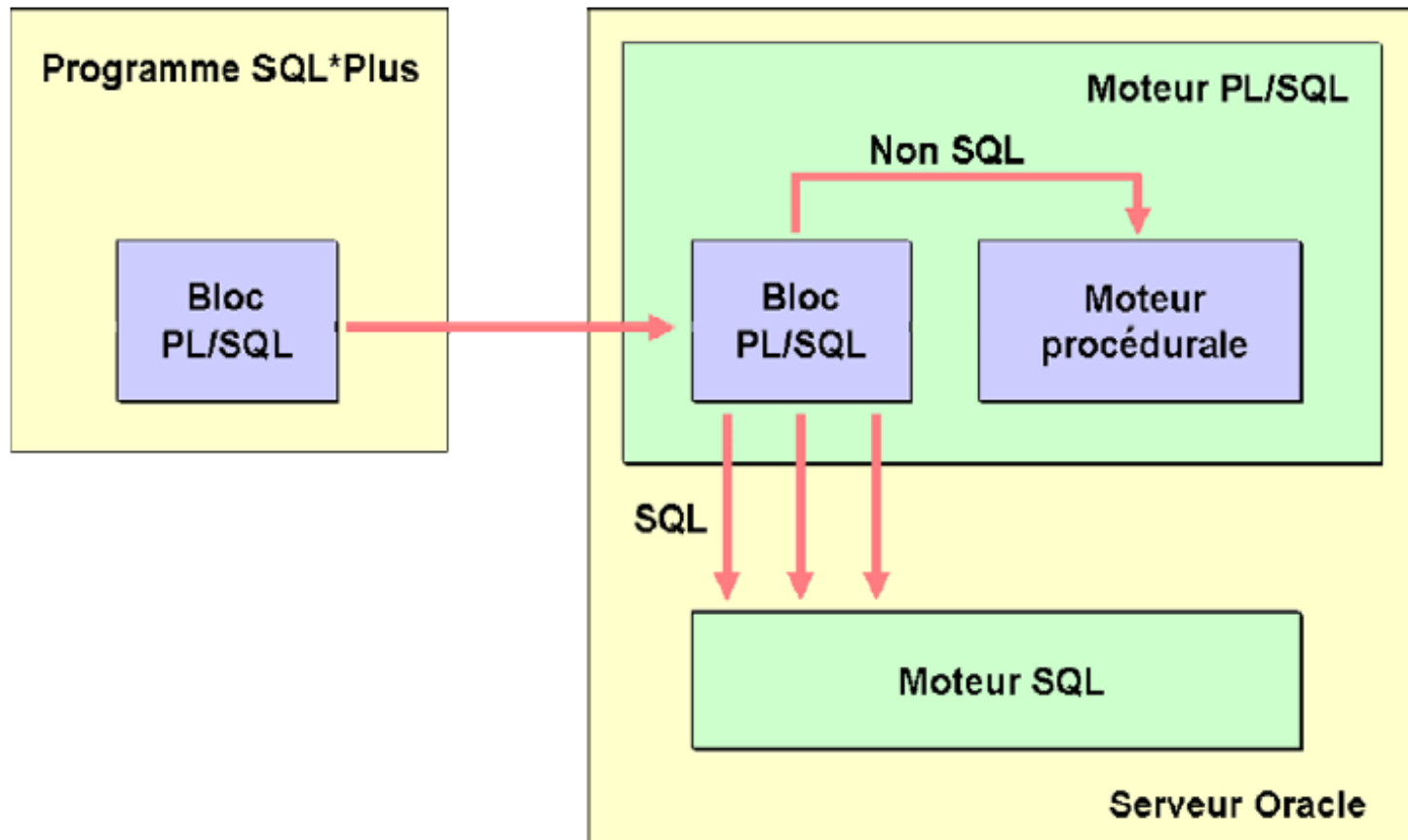
F. BARIKA KTATA

- PL/SQL comprend :
 - La partie LID de SQL (SELECT)
 - La partie LMD de SQL (UPDATE, INSERT, DELETE)
 - La gestion de transaction (COMMIT, ROLLBACK, SAVEPOINT)
 - Les fonctions de SQL
 - Plus une partie Procédurale (IF, WHILE,...)

→ PL/SQL est un **langage algorithmique complet**.

Environnement

F. BARIKA KTATA



Environnement

F. BARIKA KTATA

- Le fonctionnement de PL/SQL est basé sur l'interprétation d'un **BLOC** de commandes.
- ➔ Gain de transmission et performance.

Structure d'un bloc

- La structure d'un bloc PL/SQL est la suivante :

Section déclarative	Elle contient toutes les déclarations des variables, constantes, curseurs et exceptions définies par l'utilisateur et utilisées dans la section exécutable. La section déclarative est facultative et doit commencer par le mot clé DECLARE .
Section exécutable	Elle contient des instructions SQL permettant de manipuler les données de la base de données et des instructions PL/SQL permettant de manipuler les données d'un bloc. La section exécutable est obligatoire et doit commencer par le mot clé BEGIN .
Section de traitement des exceptions	Elle spécifie les actions à effectuer lorsque des erreurs ou des conditions anormales se produisent dans la section exécutable. La section de traitement des exceptions est facultative et doit commencer par le mot clé EXCEPTION .

Structure d'un bloc

F. BARIKA KTATA

[DECLARE]

[<Section déclarative optionnelle>]

BEGIN

<Section exécutable obligatoire>

[EXCEPTION]

[<Section de traitement des exceptions optionnelle>]

END;

Exemples :

DECLARE

v_ename VARCHAR2(5);

BEGIN

SELECT ename INTO v_ename FROM emp WHERE empno = 1234;

EXCEPTION

WHEN NO_DATA_FOUND THEN ...

END;

Declaration PL/SQL

F. BARIKA KTATA

- La partie déclarative est délimitée par le mot clef **DECLARE** qui spécifie le début et le mot clef **BEGIN** qui spécifie la fin.
- Les types de déclaration possible dans cette partie sont les suivants :
 - Variables et constantes.
 - Curseurs.
 - Exception.

Variable et types de données

- Une variable est un espace de stockage de données :

Variable_name [**CONSTANT**] datatype [**NOT NULL**][:=**DEFAULT** expr];

datatype :	le type de données de la variable.
CONSTANT :	Ce mot clé contraint la variable à contenir une valeur qui ne peut être changée.
NOT NULL :	ces mots clés font que la variable doit contenir une valeur.
expr :	l'expression définissant la valeur par défaut d'une variable peut être une valeur littérale, une autre variable ou une expression impliquant des opérateurs et des fonctions.

Variable et types de données

- PL/SQL prend en charge 4 type de données :

Scalaire	Contient une valeur unique.
Composé	Les variables de type composé sont des variables temporaires auxquelles différentes valeurs peuvent être affectées au cours de l'exécution. Ex : types RECORD, TABLE.
Référence	Il s'agit d'un pointeur vers un autre type de données. Il contient des valeurs appelées pointeurs, désignant d'autres objets programme.
LOB	Il s'agit d'un "localisateur" qui spécifie la localisation de gros objet (des images, des vidéos...).

Les types de données scalaire

NUMBER	INTEGER		Entier.
	POSITIVE		Entier positif
	BINARY_INTEGER		Entiers entre -2 147 483 647 et 2 147 483 947.
	NUMBER [(précision, degré)]		Nombre flottant
	PLS_INTEGER		Entiers signés entre -2 147 483 647 et 2 147 483 947.
CHAR	LONG	Chaîne de caractères variables (jusqu'à 32 760 octets).	
	VARCHAR2 (L)	Chaîne de caractères variables (jusqu'à 32 767 octets)	
	CHAR [(maximum_length)]		Chaîne de caractère fixe (jusqu'à 32767 octets, si maximum_length n'est pas spécifié, alors sa taille est 1).
DATE .		Date et heure.	
BOOLEAN		Type de données pouvant stocker trois valeurs possibles utilisées pour les calculs logiques : TRUE, FALSE ou NULL	
RAW	RAW (maximum_size)		Type de données stockant des données binaires. Sa taille maximale est de 2 000 octets.
	LONG RAW (maximum_size)		Type de données stockant des données binaires. Sa taille maximale est de 32 760 octets..

Les types de données scalaire

F. BARIKA KTATA

- Exemples :

v_location VARCHAR2(13) := 'Atlanta' ;

v_deptno NUMBER(2) NOT NULL := 10 ;

c_comm CONSTANT NUMBER DEFAULT 100 ;

v_hiredate DATE := SYSDATE + 7 ;

v_valid BOOLEAN NOT NULL := TRUE ;

v_count BINARY_INTEGER := 0 ;

Les types de données scalaire

- Déclaration de variable scalaire à l'aide de **%TYPE** :

Nom_variable nom-table.nom_colonne%TYPE
[NOT NULL][:= | DEFAULT expr];

Exemple :

v_ename emp.ename%TYPE ;

Les types de données composées

- Les Records sont des enregistrement PLSQL pouvant stocké une ligne constitué de colonne ayant chacune son propre nom et type.

- Syntaxe :

```
TYPE nom_record IS RECORD  
( colone1{type|variable%TYPE|  
table.colon%TYPE|table%ROWTYPE} [NOT  
NULL {:=|DEFAULT}expr],...);
```

Les types de données composées

F. BARIKA KTATA

- On peut aussi déclarer un record directement à l'aide de %ROWTYPE.
- %ROWTYPE est un attribut utilisé pour déclarer un enregistrement PLSQL pouvant stocker une ligne entière de donnée sélectionnée dans une table ou une vue.
- Syntaxe :
Nom_record nom_table%**ROWTYPE**;

Extraction de données

F. BARIKA KTATA

- Pour traiter les données stocké dans une table de la BD, il faut tous d'abord les extraire et les placées dans des variables PL/SQL.
- Cette opérations peut être effectuée par l'instruction :

```
SELECT select_list  
INTO { variable_name[,variable_name,...] |  
      record_name }  
FROM table_list  
[WHERE condition] ;
```


Erreur générer par SELECT...INTO

- L'instruction SELECT ne doit extraire qu'une seule ligne, sinon une exception ce produit.
- Deux types d'erreur peuvent être généré par l'ordre SELECT...INTO :
 - TOO_MANY_ROWS : si le SELECT retourne plusieurs valeurs.
 - NO_DATA_FOUND : si le SELECT ne retourne aucune valeur.

Package DBMS_OUTPUT

F. BARIKA KTATA

Permet de faire sortir des messages et des valeurs de blocs PL/SQL.

Fonction/Procédure	Description
PUT	Ajoute le texte de la procédure à la ligne courante du buffer de sortie.
NEW_LINE	Place un marqueur End_Of_Line dans le buffer de sortie.
PUT_LINE	Combine l'action de PUT et de NEW_LINE.
GET_LINE	Récupère la ligne courante du buffer de sortie dans la procédure.
GET_LINES	Récupère une série de lignes du buffer de sortie dans la procédure.
ENABLE/DISABLE	Active ou désactive les appels à la procédure DBMS_OUTPUT

Structures de contrôle

F. BARIKA KTATA

- Traitement conditionnel :

IF *condition* THEN *statements* ;
[ELSIF *condition* THEN *statements* ;]
[ELSE *statements* ;]
END IF ;

Structures de contrôle

F. BARIKA KTATA

- Traitement conditionnel :
 - La condition peut utiliser les variables définies ainsi que tous les opérateurs présents dans SQL (=, <, >; >=, <=, <>, IS NULL, IS NOT NULL,...)
 - L'expression peut être elle aussi une structure conditionnelle.

exemple

F. BARIKA KTATA

```
DECLARE
  vjob emp.job%TYPE;
  vnom emp.ename%TYPE := 'MILLER';
  message VARCHAR2(30);
BEGIN
  SELECT job INTO vjob FROM emp WHERE ename=vnom;
  IF vjob='VENDEUR' THEN
    UPDATE emp SET comm=1000 WHERE ename=vnom;
    message:=vnom||'a 1000 Dinars de commission';
  ELSE
    UPDATE emp SET comm=0 WHERE ename=vnom;
    message:=vnom||'pas de commission';
  END IF;
  INSERT INTO resultat VALUES (vjob, vnom, message);
  COMMIT;
END;
/
```

1. L'instruction LOOP

```
LOOP  
    statement1 ;  
    statement2 ;  
    ..  
    statementN ;  
END LOOP ;
```

➔ De cette façon la boucle est infinie, afin de sortir de la boucle on utilise l'instruction EXIT ou EXIT WHEN *condition*.

Exemple

F. BARIKA KTATA

```
DECLARE
    nbr INTEGER;
BEGIN
    nbr:=0;
    LOOP
        nbr:=nbr+1;
        IF nbr>10
            THEN EXIT;
        END IF;
    END LOOP;
END;
```

```
DECLARE
    nbr INTEGER;
BEGIN
    nbr:=0;
    LOOP
        nbr:=nbr+1;
        EXIT WHEN nbr>10;
    END LOOP;
END;
```

Structures de controle

F. BARIKA KTATA

2. L'instruction FOR...LOOP

FOR compteur **IN** [**REVERSE**] var_deb..var_fin **LOOP**

...

statements ;

...

END LOOP ;

- compteur est une variable entière locale à la boucle dont la valeur de départ est égale par défaut à var_deb et s'incrémente de 1 jusqu'à atteindre var_fin.

- **REVERSE** : permet de faire une décrémentation de 1.

Exemple

F. BARIKA KTATA

```
DECLARE
  id INTEGER:=0;
BEGIN
  FOR i IN 1..10 LOOP
    INSERT INTO res VALUES (id);
    id:=id+i;
  END LOOP;
END;
```

3. L'instruction WHILE...LOOP

```
WHILE condition LOOP  
    instruction1 ;  
    ...  
    instructionN ;  
END LOOP ;
```

Exemple

- Soit le schéma relationnel suivant :
Emp (empno, ename, job, #mgr, hiredate, sal, comm, #deptno)
Dept (deptno, dname, loc)
- Trouver le nom et le salaire des employés ayant une matricule comprise entre 7900 et 7999.

```
DECLARE
    salaire emp.sal%TYPE;
    nom emp.ename%TYPE;
    matricule NUMBER(4):=7900;
BEGIN
    WHILE matricule<8000 LOOP
        SELECT sal, ename INTO salaire, nom FROM emp
        WHERE empno=matricule;
        matricule :=matricule+1;
        INSERT INTO resultat (sal, ename) VALUES (salaire, nom);
    END LOOP;
    COMMIT;
END;
```

Gestion des exceptions

F. BARIKA KTATA

- Le mécanisme de gestion des erreurs dans PL/SQL est appelé gestionnaire des exceptions.
- Il permet d'affecter des traitements appropriés aux erreurs apparues dans un bloc PL/SQL.
- C'est pourquoi on distingue 2 types d'erreurs ou d'exceptions :
 - Erreur détectée par oracle
 - Erreur ou anomalie définie par l'utilisateur

Erreur détecté par oracle

Nom de l'exception	Num de l'erreur du serveur Oracle	Description
ACCESS_INTO_NULL	ORA-06530	Des valeurs ont essayé d'être assignées à des attributs d'un objet non initialisé
COLLECTION_IS_NULL	ORA-06531	Des méthodes autres que EXISTS ont essayé d'être appliquées à un tableau ou une table imbriquée.
CURSOR_ALREADY_OPEN	ORA-06511	Ouverture d'un curseur ouvert.
DUP_VAL_ON_INDEX	ORA-00001	Une valeur existante a essayé d'être insérée.
INVALID_CURSOR	ORA-01001	L'opération sur un curseur illégale s'est faite.
INVALID_NUMBER	ORA-01722	La conversion de chaînes de caractères en nombre a échoué.

Erreur détecté par oracle

Nom de l'exception	Num de l'erreur du serveur Oracle	Description
LOGIN_DENIED	ORA-01017	La connexion à Oracle a été réalisé avec un nom d'utilisateur invalide ou un mot de passe.
NO_DATA_FOUND	ORA-01403	Un ordre SELECT ne retourne aucune donnée.
NOT_LOGGED_ON	ORA-01012	Un programme PL/SQL interroge une base de données sans être connecté à Oracle.
PROGRAM_ERROR	ORA-06501	Le moteur PL/SQL a un problème interne.
ROWTYPE_MISMATCH	ORA-06504	La variable d'un curseur hôte et la variable d'un curseur invoquées dans un assignement ont des types de données retournés invalides.
STORAGE_ERROR	ORA-06500	Il n'y a pas assez de mémoire pour le moteur PL/SQL ou la mémoire est corrompue.

Erreur détecté par oracle

Nom de l'exception	Num de l'erreur	Description
SUBSCRIPT_BEYOND_COUNT	ORA-06533	L'élément d'une table imbriquée ou d'un tableau a été référencé en utilisant un numéro d'index plus grand que le nombre d'éléments contenu dans la collection.
SUBSCRIPT_OUTSIDE_LIMIT	ORA-06532	L'élément d'une table imbriquée ou d'un tableau a été référencé en utilisant un numéro d'index qui est en dehors des limites légales (-1 par exemple).
TOO_MANY_ROWS	ORA-01422	Un ordre SELECT retourne plus d'une ligne.
VALUE_ERROR	ORA-06502	Une erreur arithmétique, de conversion, de troncation ou de contrainte de taille est apparu.
ZERO_DIVIDE	ORA-01476	Une division par zéro a été tenté.

Erreur détecté par oracle

- Pour capturer ces exceptions on utilise la partie **EXCEPTION** du bloc PLSQL :

```
DECLARE
    --déclarations des variables
BEGIN
    --traitements
EXCEPTION
    WHEN exception1 [OR exception2...] THEN
        traitement;
    WHEN exception3 THEN traitement;
    WHEN OTHERS THEN traitement;
END;
```


Erreur ou anomalie prévue par l'utilisateur

- Ce sont des erreurs définies par l'utilisateur correspondant à des conditions que le développeur a jugé comme anormale.
- 1. Elles sont définies dans la section déclarative par le mot clef EXCEPTION.
- 2. Elles sont levées explicitement dans la partie exécutive.
- 3. Elles sont référencé dans une routine de la section de gestion des exceptions.

Exemple

```
DECLARE
    salaire emp.sal%TYPE;
BEGIN
    SELECT sal INTO salaire FROM emp;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Pas de données
        retrouv  ');

    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Nombre de donn  e
        retrouv   sup  rieur a un');
END;
```

Exemple

F. BARIKA KTATA

```
DECLARE
    mon_exception EXCEPTION;
    salaire emp.sal%TYPE;
BEGIN
    SELECT sal INTO salaire FROM emp WHERE empno=2000;
    IF salaire>2000 THEN
        RAISE mon_exception;
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Pas de données retrouvé');

    WHEN mon_exception THEN DBMS_OUTPUT.PUT_LINE('mon
    exception a été levé ');
END;
```

La procedure

RAISE_APPLICATION_ERROR

F. BARIKA KTATA

- Syntaxe :

RAISE_APPLICATION_ERROR

(num_erreur , message [, {TRUE | FALSE }])

- num_erreur : numéro de l'exception spécifié par le développeur entre -20000 et -20999.
- message : message d'erreur associé à l'exception et spécifié par le développeur. 2048 octet maximum.
- TRUE|FALSE :si TRUE l'erreur est placée sur la pile des erreurs précédentes. Sinon l'erreur remplace toutes les erreurs précédentes.

Exemple

F. BARIKA KTATA

```
DECLARE
    salaire emp.sal%TYPE;

BEGIN
    SELECT sal INTO salaire FROM emp WHERE empno=2000;
    IF salaire>2000 THEN
        RAISE_APPLICATION_ERROR(-20201,'ma deuxième façon
de traite les exceptions');
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Pas de données retrouvé');
END;
```

Exercices

F. BARIKA KTATA

1. Ecrire un bloc pl/sql qui permet d'afficher les n premiers salariés dans la table emp.
2. Ecrire un bloc pl/sql qui permet de mettre à jour un employé donné en lui augmentant sa commission de 10%.
3. Ecrire un bloc pl/sql qui permet de calculer la moyenne des salaires des employés et d'afficher les salariés qui perçoivent plus que la moyenne.